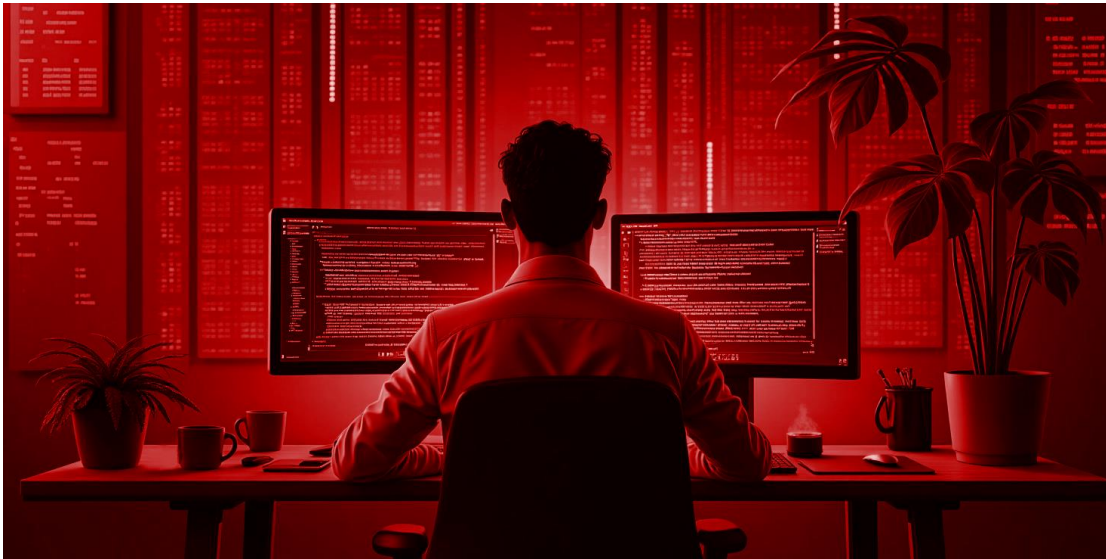




Universidad de El Salvador

Facultad Multidisciplinaria de Occidente – Ingeniería en Desarrollo de Software

Asignatura: Diseño y Programación de Bases de Datos GT1	Unidad #: 2	Tema: Álgebra relacional - Integridad referencial y Manipulación de datos
	Fecha:	Tutor:

Introducción:

Bienvenidos al material de lectura 2 de la Unidad #2 de la materia Diseño y Programación de Bases de Datos. En el ámbito de las bases de datos relacionales, las operaciones de conjunto como intersección y diferencia permiten manipular y analizar datos de manera eficiente. Estas operaciones resultan esenciales para identificar coincidencias y diferencias entre conjuntos de datos, lo que facilita la extracción de información relevante en diversos contextos.

Este documento explora el uso de las operaciones INTERSECT y EXCEPT, aplicadas a consultas SQL sobre asignaturas impartidas en distintos semestres. Se analizará el comportamiento de estas operaciones, la eliminación y conservación de duplicados mediante INTERSECT ALL y EXCEPT ALL, y su impacto en los resultados obtenidos.

Además, se aborda la creación de tablas en bases de datos, explicando las consideraciones clave al definir una estructura de datos, como la elección de nombres de tablas y columnas, los tipos de datos y las restricciones de integridad. Se presentan ejemplos prácticos con la sentencia CREATE TABLE, destacando el uso de claves primarias, claves foráneas y restricciones CHECK para garantizar la consistencia de los datos.

[illegible]

ECE-101 en el semestre de primavera de 2010, entonces habría dos tuplas con ECE-101 en el resultado.

asignatura_id
CS-101

Figura 2.4.5 Relación resultado de $c1 \text{ intersect } c2$.

2.1.2 EXCEPTO

Para encontrar todas las asignaturas que se enseñaron en el semestre de otoño de 2009 pero que no se enseñaron en el semestre de primavera de 2010, escribimos:

```
(select asignatura_id
from sección
where semestre = 'Otoño' and año = 2009)
except
(select asignatura_id
from sección
where semestre = 'Primavera' and año = 2010);
```

El resultado de esta consulta se muestra en la **Figura 2.4.6**. Fíjese en que es exactamente la relación $c1$ de la **Figura 2.4.2**, excepto que la tupla de CS-101 no aparece.

La operación `except` da como salida todas las tuplas de su primera entrada que no existen en la segunda; es decir, realiza la diferencia. Por ejemplo, si se diese el caso de que se enseñasen cuatro secciones de la asignatura ECE-101 en el semestre de otoño de 2009 y dos secciones de la asignatura ECE-101 en el semestre de primavera de 2010, el resultado de la operación `except` no tendría ninguna copia de ECE-101.

Si se quisiesen mantener los duplicados, se debería utilizar `except all` en lugar de `except`:

```
(select asignatura_id
from sección
where semestre = 'Otoño' and año = 2009)
except all
(select asignatura_id
from sección
where semestre = 'Primavera' and año = 2010);
```

El número de copias duplicadas de una tupla en el resultado es igual al número de copias duplicadas en c1 menos el número de copias duplicadas en c2, siempre que esta diferencia sea positiva.

Por tanto, si se diese el caso de que se enseñasen cuatro secciones de la asignatura ECE-101 en el semestre de otoño de 2009 y dos secciones de la asignatura ECE-101 en el semestre de primavera de 2010, entonces habría dos tuplas con ECE-101 en el resultado. Sin embargo, si hubiese dos o menos secciones de ECE-101 en el semestre de otoño de 2009 y dos secciones en el semestre de primavera de 2010, no quedaría ninguna tupla en el resultado.

asignatura_id
CS-347
PHY-101

Figura 2.4.6 Relación resultado de c1 except c2.

2.2 Creación de tabla de datos

Empezaremos creando una tabla. Antes de hacerlo es conveniente planificar ciertos aspectos:

- El nombre de la tabla. Debe ser un nombre que identifique su contenido. Por ejemplo, llamamos a una tabla ALUMNOS porque contendrá datos sobre alumnos.
- El nombre de cada columna de la tabla. Ha de ser un nombre autodescriptivo, que identifique su contenido. Por ejemplo, DNI, NOMBRE o APELLIDOS.
- El tipo de dato y el tamaño que tendrá cada columna.
- Las columnas obligatorias, los valores por defecto, las restricciones, etcétera.

La denominación de la tabla puede tener de 1 a 30 caracteres de longitud. Ha de ser única y no puede ser una palabra reservada. Su primer carácter debe ser alfabético y el resto pueden ser letras, números y el carácter de subrayado.

Para crear una tabla usamos la orden CREATE TABLE, cuyo formato más simple es:

```
CREATE TABLE Nombretabla
(
  Columna1 Tipo_dato [NOT NULL],
  Columna2 Tipo_dato [NOT NULL],
  .....
) [TABLESPACE espacio_de_tabla];
```

Donde:

- **Columna1, Columna2:** son los nombres de las columnas que contendrá cada fila.
- **Tipo_dato:** indica el tipo de dato (VARCHAR2, NUMBER, etcétera) de cada columna.
- **TABLESPACE espacio_de_tabla:** señala el TABLESPACE para almacenar la tabla.
- **NOT NULL:** indica que la columna debe contener alguna información; nunca puede ser nula.

Cuando almacenamos datos en nuestras tablas, se ajustan a una serie de restricciones predefinidas. Por ejemplo, que una columna no pueda almacenar valores negativos, que una cadena de caracteres se deba almacenar en mayúsculas o que una columna no pueda ser 0.

La integridad hace referencia al hecho de que los datos de la base de datos han de ajustarse a restricciones antes de almacenarse en ella. Así pues, una restricción de integridad será una regla que restringe el rango de valores para una o más columnas en la tabla.

Si se produce cualquier fallo mientras un usuario está cambiando los datos en la base de datos, ésta tiene la capacidad de deshacer o cancelar cualquier transacción sospechosa.

Existe otro tipo de integridad, que es la integridad referencial, la cual garantiza que los valores de una columna (o columnas) de una tabla (clave ajena) dependan de los valores de otra columna (o columnas) de otra tabla (clave primaria).

2.3 Restricciones

La orden CREATE TABLE permite definir distintos tipos de restricciones sobre una tabla: claves primarias, claves ajenas, obligatoriedad, valores por defecto y verificación de condiciones.

Para definir las restricciones en la orden CREATE TABLE usamos la cláusula CONSTRAINT.

This image shows a vertical rectangular sheet of white paper designed for handwriting practice. It features ten sets of horizontal dashed lines, evenly spaced from top to bottom. Each set consists of three parallel dashed lines, providing a guide for letter height and placement. The paper is otherwise blank, with no margins, text, or other markings.

Las restricciones de la orden CREATE TABLE que aparecen al final de la definición de las columnas (o de tabla) se diferencian de la anterior en que se puede hacer referencia a varias columnas en una única restricción (por ejemplo, declarando dos columnas como clave primaria o ajena):

```
CREATE TABLE nombre_tabla
(
  Columna1 TIPO_DE_DATO [NOT NULL],
  Columna2 TIPO_DE_DATO [NOT NULL],
  Columna3 TIPO_DE_DATO [NOT NULL],
  ...
  [CONSTRAINT nombre_restricción]
  { [UNIQUE] | [PRIMARY KEY] ( columna [ , columna] ) },
  [CONSTRAINT nombre_restricción]
  [FOREIGN KEY ( columna [ , columna] )
  REFERENCES Nombretabla [ ( columna [ , columna] ) ]
  [ON DELETE CASCADE] ],
  [CONSTRAINT nombre_restricción]
  [CHECK (condición) ],
  ...
) [TABLESPACE espacio_de_tabla];
```

2.3.1 Clave primaria. La restricción PRIMARY KEY

Una clave primaria dentro de una tabla es una columna o un conjunto de columnas que identifican unívocamente a cada fila. Debe ser única, no nula y obligatoria. Como máximo podemos definir una clave primaria por tabla. Esta clave se puede referenciar por una columna o columnas de otra tabla. Llamamos clave ajena a esta columna o columnas.

Cuando se crea una clave primaria, automáticamente se crea un índice que facilita el acceso a la tabla. Para definir una clave primaria en una tabla usamos la restricción PRIMARY KEY.

Éstos son los formatos de la orden CREATE TABLE para definir claves primarias:

- El formato de restricción de columna es el siguiente:

```
CREATE TABLE nombre_tabla
(
  Columna1 TIPO_DE_DATO [CONSTRAINT nombrerestricción]
  PRIMARY KEY ,
  Columna2 TIPO_DE_DATO ,
  ...
) [TABLESPACE espacio_de_tabla];
```


- Y el formato de restricción de tabla es éste:

```
CREATE TABLE nombre_tabla
(
  Columna1 TIPO_DE_DATO,
  Columna2 TIPO_DE_DATO,
  ...
  [CONSTRAINT nombre_restricción] PRIMARY KEY (columna [ , columna ] ) ,
  ...
) [TABLESPACE espacio_de_tabla];
```

2.3.2 Claves ajenas. Las restricciones FOREIGN KEY

Una clave ajena está formada por una o varias columnas que están asociadas a una clave primaria de otra o de la misma tabla. Se pueden definir tantas claves ajenas como sea preciso, y pueden estar o no en la misma tabla que la clave primaria. El valor de la columna o columnas que son claves ajenas debe ser NULL o igual a un valor de la clave referenciada (regla de integridad referencial).

El formato de CREATE TABLE para definir claves ajenas puede ser cualquiera de los siguientes:

- Formato de restricción de columna La clave ajena se define en la descripción de la columna usando la cláusula REFERENCES:

```
CREATE TABLE nombre_tabla
(
  Columna1 TIPO_DE_DATO
  [CONSTRAINT nombre_restricción]
  REFERENCES Nombretabla [ (columna) ] [ON DELETE CASCADE] ,
  ...
  Columna2 TIPO_DE_DATO ,
  ...
) [TABLESPACE espacio_de_tabla];
```

- Formato de restricción de tabla La clave ajena se define al final de todas las columnas empleando las cláusulas FOREIGN KEY y REFERENCES:

```
CREATE TABLE nombre_tabla
(
  Columna1 TIPO_DE_DATO ,
  Columna2 TIPO_DE_DATO ,
  ...
  [CONSTRAINT nombre_restricción]
```



```
FOREIGN KEY (columna [ , columna] )
REFERENCES Nombretabla [(columna [ , columna] ) ] [ON DELETE CASCADE]
,
...
) [TABLESPACE espacio_de_tabla];
```

En la cláusula REFERENCES indicamos la tabla a la cual remite la clave ajena. La derecha de FOREIGN KEY y, entre paréntesis, indicamos la columna o columnas que forman parte de la clave ajena.

La cláusula ON DELETE CASCADE o borrado en cascada se define cuando al borrar las filas asociadas con claves primarias deseamos que se eliminen automáticamente las filas con claves ajenas que referencian a dichas claves.

2.3.3 Obligatoriedad. La restricción Not Null

Esta restricción asociada a una columna significa que no puede tener valores nulos, es decir, que ha de tener obligatoriamente un valor. En caso contrario, causa una excepción. En ejemplos anteriores nos hemos ocupado de cómo se define una columna con la restricción NOT NULL. Éste es su formato:

```
CREATE TABLE nombre_tabla
(
Columna1 TIPO_DE_DATO [CONSTRAINT nombre_restricción]
NOT NULL ,
Columna2 TIPO_DE_DATO
...
) [TABLESPACE espacio_de_tabla];
```

2.3.4 Valores por defecto. La especificación DEFAULT

En el momento de crear una tabla podemos asignar valores por defecto a las columnas.

Si especificamos la cláusula DEFAULT a una columna, le proporcionamos un valor por omisión cuando el valor de la columna no se especifica en la cláusula INSERT.

En la especificación DEFAULT es posible incluir varias expresiones: constantes, funciones SQL y variables UID y SYSDATE. No se puede hacer referencias a columnas o a funciones PL/SQL. Ejemplo:

```
CREATE TABLE EJEMPL01
(
DNI VARCHAR2(10) NOT NULL,
```

```
NOMBRE VARCHAR(30),  
FECHA DATE DEFAULT SYSDATE  
);
```

2.3.5 Verificación de condiciones. La restricción CHECK

Muchas columnas de tablas requieren valores limitados dentro de un rango o el cumplimiento de ciertas condiciones. Con una restricción de verificación de condiciones se puede expresar una condición que ha de cumplirse para todas y cada una de las filas de la tabla.

La restricción CHECK actúa como una cláusula WHERE. Puede hacer referencia a una o a más columnas, pero no a valores de otras filas. En una cláusula CHECK no cabe incluir subconsultas ni las pseudocolumnas SYSDATE, UID y USER.

Éstos son los formatos con la orden CREATE TABLE con la restricción CHECK:

- Forma de restricción de columna:

```
CREATE TABLE nombre_tabla  
(  
  Columna1 TIPO_DE_DATO [CONSTRAINT nombrerestricción]  
  CHECK (condición),  
  Columna2 TIPO_DE_DATO  
  ...  
) [TABLESPACE espacio_de_tabla];
```

- Formato de restricción de tabla

```
CREATE TABLE nombre_tabla  
(  
  Columna1 TIPO_DE_DATO ,  
  Columna2 TIPO_DE_DATO ,  
  ...  
  [CONSTRAINT nombrerestricción] CHECK (condición),  
  ...  
) [TABLESPACE espacio_de_tabla];
```

2.3.6 La restricción UNIQUE

Evita valores repetidos en la misma columna. Puede contener una o varias columnas. Es similar a la restricción PRIMARY KEY, salvo que son posibles varias columnas UNIQUE definidas en una tabla. Admite valores NULL. Al igual que en PRIMARY KEY, cuando se define una restricción UNIQUE se crea un índice automáticamente. Veamos su formato:

- Formato de restricción de columna:

[illegible]

Ejemplo 1. Insertar un solo registro, con todos los campos especificados. Operación: Insertar un nuevo registro Faculty con ID de F330, nombre Jones, departamento CSC y clasificación de Instructor.

Faculty			
facId	name	department	rank
F101	Adams	Art	Professor
F105	Tanaka	CSC	Instructor
F110	Byrne	Math	Assistant
F115	Smith	History	Associate
F221	Smith	CSC	Professor

Tabla 2.7.1 Tabla Faculty.

Comando SQL:

```
INSERT
INTO Faculty (facId, name, department, rank)
VALUES ('F330', 'Jones', 'CSC', 'Instructor');
```

Ejemplo 2. Insertar un solo registro, sin especificar campos

Operación: Insertar un nuevo registro de estudiante con ID de S1030, nombre Alice Hunt, especialidad art y 12 créditos

Student				
stuld	lastName	firstName	major	credits
S1001	Smith	Tom	History	90
S1002	Chin	Ann	Math	36
S1005	Lee	Perry	History	3
S1010	Burns	Edward	Art	63
S1013	McCarthy	Owen	Math	0
S1015	Jones	Mary	Math	42
S1020	Rivera	Jane	CSC	15

Tabla 2.7.2 Tabla Student.

Consulta SQL:

```
INSERT
INTO Student
VALUES ('S1030', 'Hunt', 'Alice', 'Art', 12);
```

Note que no fue necesario especificar nombres de campo, porque el sistema supone que se quieren todos los campos en la tabla. Podría hacer lo mismo para el ejemplo anterior.

Ejemplo 3. Insertar un registro con valor nulo en un campo. Operación: Insertar un nuevo registro de estudiante con ID de S1031, nombre Maria, Bono, cero créditos y sin especialidad.

Comando SQL:

```
INSERT
INTO Student (lastName, firstName, stuld, credits)
VALUES ('Bono', 'Maria', 'S1031', 0);
```

Note que se reordenaron los nombres de campo, pero no hay confusión porque se entiende que el orden de los valores coincide con el orden de los campos mencionados en el INTO, sin importar su orden en la tabla. Note también que el cero es un valor real para credits, no un valor nulo. El campo mayor se establecerá en nulo, pues se excluye de la lista de campos en la línea INTO.

Ejemplo 4. Insertar múltiples registros. Operación: Crear y llenar una nueva tabla que muestre cada curso y el número de estudiantes inscritos en ellos.

Enroll		
stuld	classNumber	grade
S1001	ART103A	A
S1001	HST205A	C
S1002	ART103A	D
S1002	CSC201A	F
S1002	MTH103C	B
S1010	ART103A	
S1010	MTH103C	
S1020	CSC201A	B
S1020	MTH101B	A

Tabla 2.7.3 Tabla Enroll.

Comando SQL:

```
CREATE TABLE Enrollment
(classNumber CHAR(7) NOT NULL,
Students SMALLINT);
INSERT
INTO Enrollment (classNumber, Students)
SELECT classNumber, COUNT(*)
FROM Enroll
GROUP BY classNumber;
```

Aquí se creó una nueva tabla, Enrollment, y se llenó al tomar datos de una tabla existente, Enroll. Si Enroll es como aparece en la **Tabla 2.7.3**, Enrollment ahora se ve como esto:

Enrollment	classNumber	Students
	ART103A	3
	CSC201A	2
	MTH101B	1
	HST205A	1
	MTH103C	2

Tabla 2.7.4 Tabla Enrollment.

La tabla Enrollment (**Tabla 2.7.4**) ahora está disponible para que el usuario la manipule, tal como lo sería cualquiera otra tabla. Se puede actualizar según se requiera, pero no se actualizará automáticamente cuando Enroll se actualice.